

①

ALGORITMA ANALİZİ

13:15
başlangıç

- Mine Elif KARSLIĞIL-

Kaynaklar

The Design & Analysis of Algorithm, A. Levitin

baş ucu kitabı gibi
(hemen her temel konu var.)

Algorithms : 4th Edition, R. Sedgewick, K. Wayne

Princeton Uni.
hocaları yazmış.

The Algorithm Design Manual, S. Skiena

(Hoca en çok bun-
dan yararlanıyor)

Hikaye kitabı gibi (Biraz Advanced)

Değerlendirme

-3 Ödev → %15

-Proje → %10

-2 Midterm → %45

-Final → %30

*Ödevlerde kullanılacak programlama dili C (ödevlere göre değişebilir)

eleman sayısı

elemanların veriliş sırası

- Best case

- Average case : en mantıklısı (ama hesabı zor)

- Worst case

$$\sum_{i=1}^n i = 1+2+3+\dots+n = \frac{n \cdot (n+1)}{2}$$

$$\sum_{i=1}^n i^2 = 1^2+2^2+3^2+\dots+n^2 = \frac{n \cdot (n+1) \cdot (2n+1)}{6} = \frac{1}{3} n^3$$

$$\sum_{i=1}^n \lg i = n \cdot \lg i$$

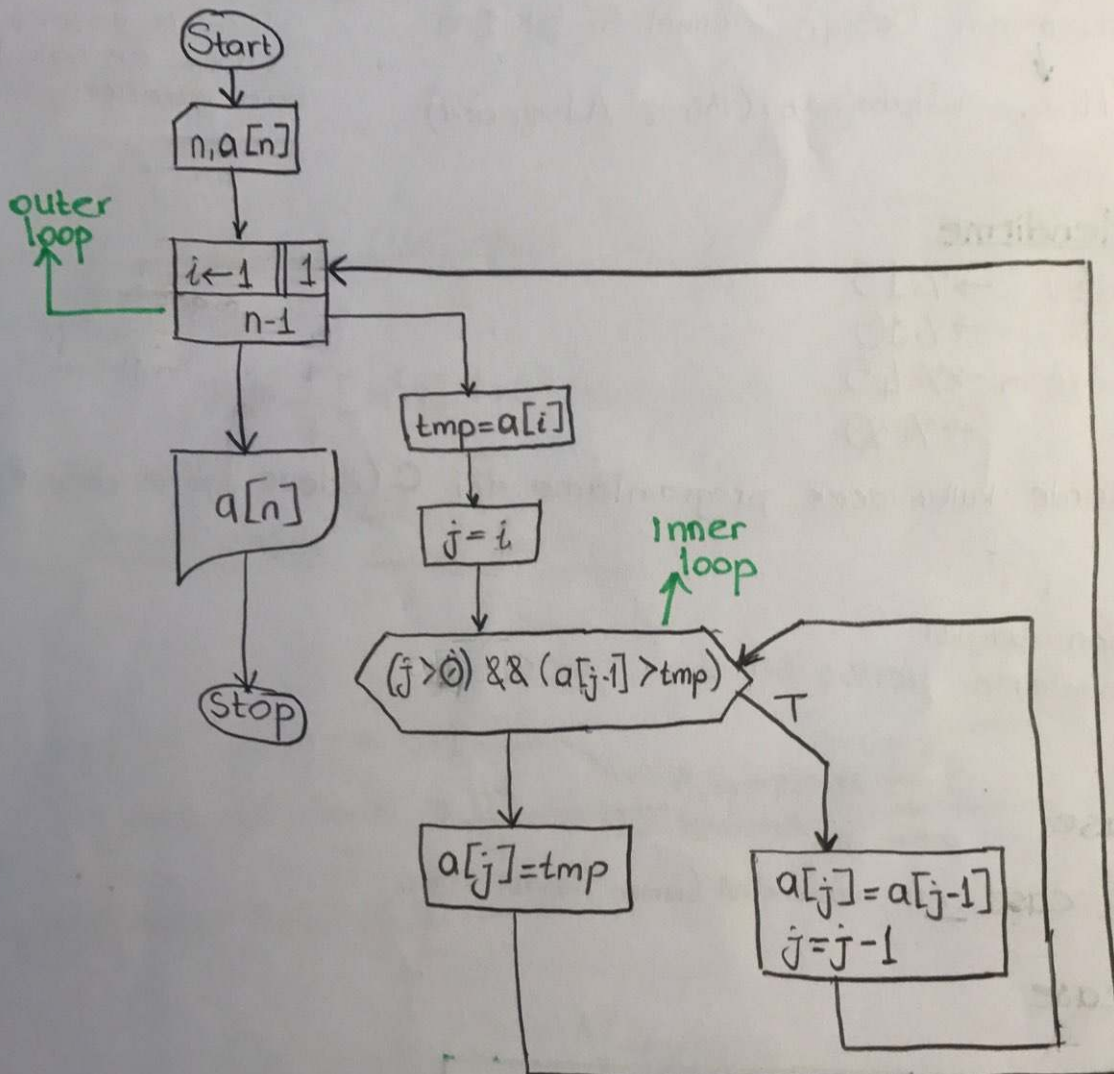


②

insertion sort

21	17	14	34	47	19	71	22	29	41	32	8
----	----	----	----	----	----	----	----	----	----	----	---

17 21 14
 14 17 17 21 34 47 19 71 22 29 41 32 8
 Sıralı Sırasız



worst case:

outer loop : 1... n-1

inner loop : $\left. \begin{matrix} j=1 \\ j=2 \\ j=3 \\ \vdots \\ j=n-1 \end{matrix} \right\} n-1$

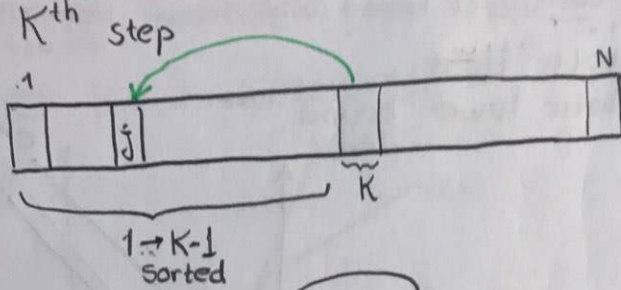
$$\rightarrow \sum_{i=1}^{n-1} \sum_{j=1}^{i-1} 1 = \sum_{j=1}^{n-1} i = \frac{n*(n-1)}{2}$$

$$\approx n^2$$



③

Average case:



inner loop

$$\frac{1}{K} \cdot \sum_{j=1}^K (K-j+1)$$

$$= \frac{1}{K} \left(K^2 - \frac{K(K+1)}{2} + K \right)$$

$$= \frac{2K^2 - K^2 - K + 2K}{2K}$$

$$\sim \frac{K+1}{2}$$

ortalama olarak
K. g. b. deki eleman
için iç döngüye gir-
me miktarı

outer loop

$$\sum_{K=2}^N \frac{K+1}{2} = \frac{1}{2} \frac{(N+1)(N+2)}{2}$$

$$= \frac{1}{4} [N^2 + 3N + 4] \Rightarrow \text{average case}$$

Best case: $\sum_{i=1}^{n-1} 1 = n-1 \approx n$

Big-OH

→ Best case $\approx n-1$

→ Average case $\approx \frac{n^2}{4}$

→ Worst case $\approx \frac{n^2}{2}$

Asymptotic
Notation

$n \rightarrow O(n)$

$n^2 \rightarrow O(n^2)$

$n^2 \rightarrow O(n^2)$

Asymptotic Notation

$$\frac{N^2}{4} + \frac{3N}{4} + \frac{1}{2}$$

$\xrightarrow[n \rightarrow \infty]{\text{sabitler elenir}} \frac{N^2}{4} + 3N + \cancel{2}$

katsayılar
elenir

N^2

Sonuç

$N^2 + \cancel{3N}$

En yüksek dere-
celi terim alınır

$\frac{N^2}{\cancel{4}} + \cancel{3N}$

Algoritmanın karmaşıklık
mertebesi

(işletim sistemi, programlama dili,
yazılım ve donanımdan bağımsız)



④

Asymptotic Notation

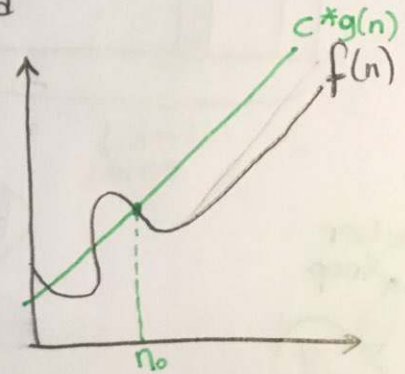
Big-O: O → asymptotic upper bound (algoritmanın karmaşıklığının en üst noktası)

Big-theta: Θ → asymptotic tight

Big-omega: Ω → asymptotic lower bound

$$f(n) = 4n^2 + 2n$$

$$\left. \begin{array}{l} f(n) \leq c * g(n) \\ n > n_0 \text{ ve } c > 0 \end{array} \right\} f(n) \text{ is } O(g(n))$$



Örnek: $f(n) = 2n + 10$
 $g(n) = n$

$$2n + 10 \leq c * n \quad n > n_0$$

$$10 \leq (c-2) * n \quad c=3 \quad n=10$$

$$f(n) = 2n + 10 \rightarrow O(n)$$

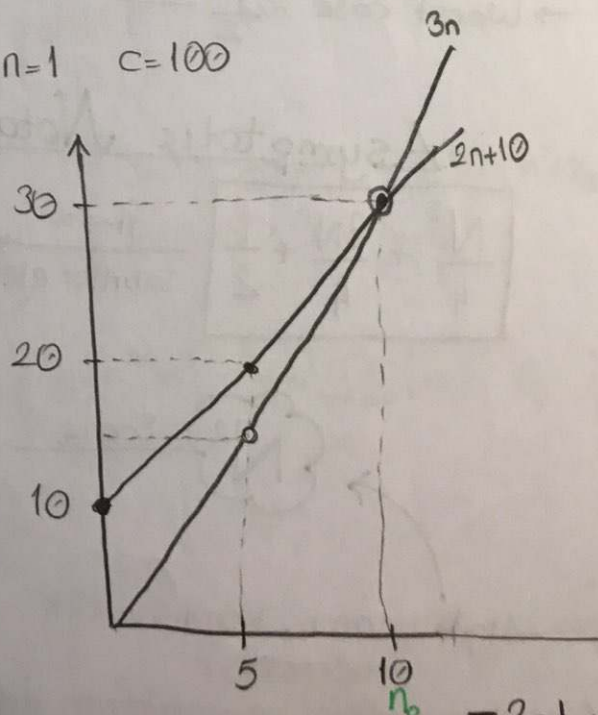
Örnek: $f(n) = 2n + 10$
 $g(n) = n^2$

$$2n + 10 \leq c * n^2 \quad n > n_0, c > 0$$

$$cn^2 - 2n \geq 10 \quad n=1 \quad c=100$$

$$f(n) = O(g(n))$$

$$2n + 10 = O(n^2)$$

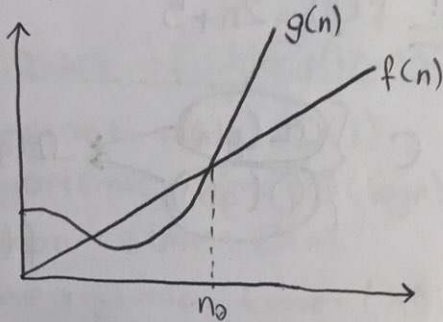


(5)

Big-Oh Notation

$f(n) \in O(g(n))$ if $0 \leq f(n) \leq c * g(n)$, $n > n_0$, $c > 0$

Örnek: $f(n) = 2n + 10 \rightarrow O(n)$



$$\begin{aligned} 2n + 10 &\leq c * n & n > n_0 \\ 10 &\leq (c - 2)n & n_0 = 1 \\ 10 &\leq c - 2 & \rightarrow 12 \leq c \end{aligned}$$

Örnek: $f(n) = 2n + 10 \rightarrow O(n^2)$

$$\begin{aligned} 2n + 10 &\leq c * n^2 & n > n_0 > 0 & n = 3 \\ 16 &\leq c * 9 & c = 2, n = 3 \end{aligned}$$

Örnek: ~~$f(n^2) \in O(n)$~~

$$\begin{aligned} n^2 &\leq c * n \\ n &\leq c \end{aligned}$$

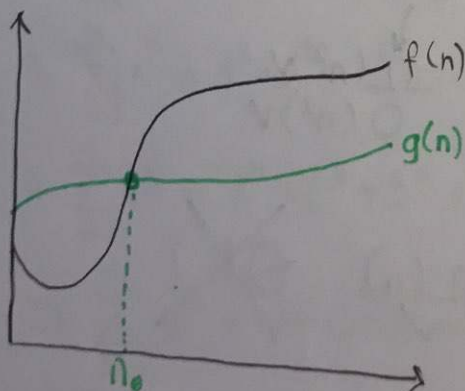
Big-Omega Notation

$f(n) \in \Omega(g(n))$ if $c > 0$, $n_0 > 0$, $n > n_0 \rightarrow f(n) \geq c * g(n)$
 asymptotic lower bound

sequential search $\rightarrow$ best case: $\Omega(1)$

insertion sort $\rightarrow$ best case: $\Omega(n)$

Örnek: $f(n) = 5n^2 \rightarrow \Omega(n^2)$



$$\begin{aligned} 5n^2 &\geq c * n^2 & c > 0, n > n_0, n_0 > 0 \\ (5 - c)n^2 &\geq 0 & n_0 = 1 \\ & & c = 1 \end{aligned}$$

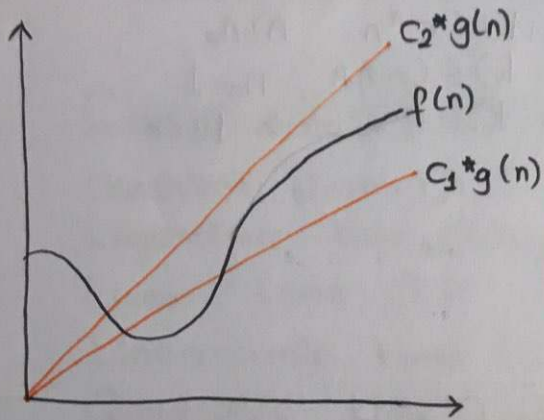
$$f(n) = 5n^2 \rightarrow \Omega(n^2) \quad ? \quad \checkmark$$



⑥

Big-theta (Θ) Notation : tight bound $\rightarrow f(x)$ is asymptotically equal to $g(x)$

$$0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \quad c_1 \leq c_2, n_0 > 0, n \geq n_0 \rightarrow f(n) = \Theta(g(n))$$



Örnek: $f(n) = 2n + 5$

$$f(n) \in \begin{cases} \Omega(g(n)) \\ O(g(n)) \end{cases} \rightarrow \Omega(g(n)) = O(g(n)) \downarrow f(n) \in \Theta(g(n))$$

$$f(n) = 2n + 5 \xrightarrow{\times} O(n^2)$$

$$f(n) \not\in \Omega(n^2)$$

$$f(n) \not\in O(1) \quad f(n) \not\in \Theta(1)$$

Örnek: $f(n) = 2n + 5 \in O(n)$

$$① \quad 2n + 5 \in \Omega(n) \rightarrow 2n + 5 \geq c * n \rightarrow 5 \geq (c - 2) * n \quad n_0 = 1, c = 3 \checkmark$$

$$② \quad 2n + 5 \in O(n) \rightarrow 2n + 5 \leq c * n \quad 5 \leq (c - 2) * n \quad n_0 = 1, c = 7 \checkmark$$

$$c_1 = 3, c_2 = 7, n_0 = 1 \rightarrow c_1 * n \leq 2n + 5 \leq c_2 * n \rightarrow 2n + 5 \in \Theta(n)$$

Örnek: Insertion Sort

$$\text{worst case} \sim n^2 \rightarrow O(n^2) \quad \checkmark \quad \Theta(n^2) \quad \checkmark$$

$$\text{best case} \sim n \rightarrow O(n) \quad \checkmark$$

$$\downarrow \quad \Omega(n^2) \quad \checkmark$$

$$\downarrow \quad O(n^2) \quad \checkmark$$

$$\downarrow \quad \Omega(n) \quad \checkmark$$

$$\downarrow \quad O(n) \quad \checkmark$$

$$\text{Insertion Sort} \rightarrow O(n^2) \quad \checkmark \quad \Omega(n) \quad \checkmark \quad \Theta(\quad) \quad \times$$



Örnek: Merge Sort

Best case $\sim n \cdot \log n$
Worst case $\sim n \cdot \log n$

$\rightarrow \Theta(n \cdot \log n)$

Classes of Algorithms

Constant time: $O(1)$

Logarithmic time: $O(\log n)$

Linear time: $O(n)$

Linearithmic time: $(n \cdot \log n)$

Quadratic time: $O(n^2)$

Cubic time: $O(n^3)$

Exponential time: $O(2^n)$

$a = b + c$

binary search

loop

merge sort

double loop

triple loop

exhaustive search

Useful Formulas for the Analysis of Algorithms

- Property of Logarithms

1. $\log x^y = y \cdot \log x$

2. $\log xy = \log x + \log y$

3. $\log \frac{x}{y} = \log x - \log y$

4. $\log_a x = \log_a b \cdot \log_b x$

5. $a^{\log_b x} = x^{\log_b a}$

- Combinatorics

1. Number of permutations of n-element set:

$$P(n) = n!$$

2. Number of k-combinations of n-element set:

$$C(n, k) = \frac{n!}{k!(n-k)!}$$

3. Number of subsets of an n-element set: 2^n

- Important Summation Formulas

1. $\sum_{i=l}^u 1 = \underbrace{1+1+1+\dots+1}_{u-l+1} = u-l+1 \quad (l \leq u)$

2. $\sum_{i=1}^n i = 1+2+3+\dots+n = \frac{n \cdot (n+1)}{2} \approx \frac{1}{2} n^2$

3. $\sum_{i=1}^n i^2 = 1^2+2^2+3^2+\dots+n^2 = \frac{n \cdot (n+1) \cdot (2n+1)}{6} \approx \frac{1}{3} n^3$

4. $\sum_{i=1}^n i^k = 1^k+2^k+3^k+\dots+n^k \approx \frac{1}{k+1} \cdot n^{k+1}$

5. $\sum_{i=0}^n a^i = 1+a+a^2+\dots+a^n = \frac{a^{n+1}-1}{a-1} \quad (a \neq 1)$

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$



⑧

$$6. \sum_{i=1}^n i \cdot 2^i = 1 \cdot 2 + 2 \cdot 2^2 + \dots + n \cdot 2^n = (n-1) \cdot 2^{n+1} + 2$$

$$7. \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n + \gamma \quad (\gamma \approx 0,5772)$$

$$8. \sum_{i=1}^n \lg i \approx n \cdot \lg n$$

Sum Manipulation Rules

$$1. \sum_{i=l}^u c \cdot a_i = c \cdot \sum_{i=l}^u a_i$$

$$2. \sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i$$

$$3. \sum_{i=l}^u a_i = \sum_{i=l}^m a_i + \sum_{i=(m+1)}^u a_i \quad \text{where } (l \leq m \leq u)$$

Floor and Ceiling Formulas

$$\lfloor 3,8 \rfloor = 3$$

$$\lfloor -3,8 \rfloor = -4$$

$$\lceil 3,8 \rceil = 4$$

$$\lceil -3,8 \rceil = -3$$

$$1. x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$$

$$2. \lfloor x+n \rfloor = \lfloor x \rfloor + n \quad \text{and} \quad \lceil x+n \rceil = \lceil x \rceil + n$$

(for real x and integer n)

$$3. \lfloor n/2 \rfloor + \lceil n/2 \rceil = n$$

$$4. \lceil \lg(n+1) \rceil = \lfloor \lg n \rfloor + 1$$

Mathematical Analysis of Nonrecursive Algorithms

Örnek:

START

$N, A[N]$

$\max = A[0]$

$I \leftarrow 1 \quad \parallel \quad 1$

$N-1$

$\max$

STOP

Two operations :

$A[I] > \max$

$\max = A[I]$

Basic operation

Günkü if kontrolü her zaman yapılıyor, atama bozen oluyor.

0	1	2	3	4	5	6	7
7	1	3	2	8	4	6	5

$C(n)$: the number of times this comparison is executed.

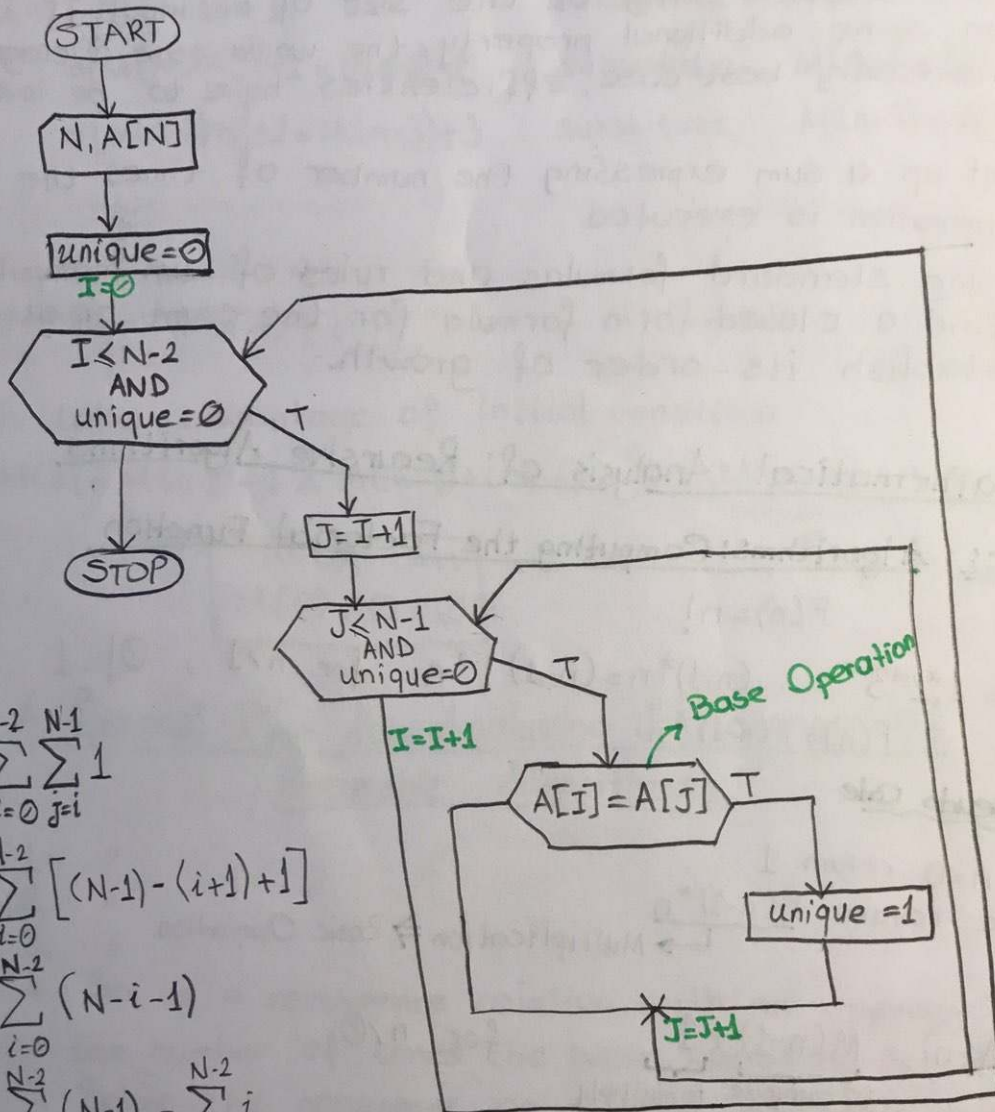
$$C(n) = \sum_{i=1}^{n-1} 1 = n-1 \quad \Theta(n)$$



Örnek: Algorithms: Element Uniqueness Problem

3 5 7 3 1 2 4

2 7 1 3 4 6 5 unique



$$\begin{aligned}
 C_{\text{worst}} &= \sum_{i=0}^{N-2} \sum_{j=i}^{N-1} 1 \\
 &= \sum_{i=0}^{N-2} [(N-1) - (i+1) + 1] \\
 &= \sum_{i=0}^{N-2} (N-i-1) \\
 &= \sum_{i=0}^{N-2} (N-1) - \sum_{i=0}^{N-2} i
 \end{aligned}$$

$$= (N-1) * \sum_{i=0}^{N-2} 1 - \frac{(N-2) * (N-1)}{2} = (N-1)^2 - \frac{(N-2) * (N-1)}{2} = \frac{N * (N-1)}{2}$$

$$\approx \frac{1}{2} N^2 \in O(N^2)$$

⑩

General Plan for Analyzing of Non-recursive Algorithms

1. Decide on a parameter (or parameters) indicating on input's size.
2. Identify the algorithm's basic operation (As a rule, it is located in its innermost loop)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case and if necessary best-case, efficiencies have to be investigated separately.
4. Set up a sum expressing the number of times the algorithm's basic operation is executed.
5. Using standard formulas and rules of sum manipulation, either find a closed-form formula for the count or, at the very least, establish its order of growth.

Mathematical Analysis of Recursive Algorithms

Example: Algorithms: Computing the Factorial Function

$$F(n) = n!$$

$$n! = 1 * 2 * 3 * \dots * (n-1) * n = (n-1)! * n \quad \text{for } n > 1, \quad 0! = 1$$

$$F(n) = F(n-1) * n!$$

Pseudo Code

if $n=0$ return 1

else return $F(n-1) * n$

→ Multiplication ⇒ Basic Operation

$$M(n) = \underbrace{M(n-1)}_{\text{to compute } F(n-1)} + \underbrace{1}_{\text{to multiply } F(n-1) \text{ by } n} \quad \text{for } n > 0$$

→ Recurrences Relations ←

-Initial Condition: It tells us the the values with which the sequence starts. We can obtain this value by inspecting the condition that makes the algorithm stop its recursive calls.

if $n=0$ return 1

$M(0) = 0$
 ↑
 the calls stop when $n=0$ → no multiplication when $n=0$



Recurrence Relation:

$$M(n) = M(n-1) + 1 \quad \text{for } n > 0$$

$$M(0) = 0$$

Method of Backward Substitution

$$M(n) = M(n-1) + 1$$

$$= [M(n-2) + 1] + 1 = M(n-2) + 2$$

$$= [M(n-3) + 1] + 2 = M(n-3) + 3$$

⋮

$$M(n) = M(n-i) + i$$

substitute $M(n-1) = M(n-2) + 1$

substitute $M(n-2) = M(n-3) + 1$

substitute $M(n-3) = M(n-4) + 1$

⋮

To take advantage of initial condition

$$M(n) = M(n-1) + 1 = M(n-2) + 2 = \dots = M(n-n) + n = n$$

$$n=0, i=n$$

$$M(0) + 0 = 0$$

A General Plan for Analyzing Efficiency of Recursive Algorithms

1. "
2. "
3. "
4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or at least ascertain the order of growth of its solution.

(12)

Example:

Input: A positive decimal integer

→ The number of binary digits in binary's representation.

if $n=1$ return 1
else return BinaryDigit $\lfloor \log_2 n \rfloor + 1$

$A(n)$: The number of Additions

$$A(n) = A(\lfloor \log_2 n \rfloor) + 1$$

Initial condition:

$$n=1 \quad A(1) = 0$$

$$n = 2^k$$

$$A(2^k) = A(2^{k-1}) + 1$$

$$A(2^0) = 0$$

Backward Substitution:

$$\begin{aligned} A(2^k) &= A(2^{k-1}) + 1 && \text{substitute} && A(2^{k-1}) = A(2^{k-2}) + 1 \\ &= [A(2^{k-2}) + 1] + 1 && \text{substitute} && A(2^{k-2}) = A(2^{k-3}) + 1 \\ &= [A(2^{k-3}) + 1] + 2 \end{aligned}$$

$$\begin{aligned} &= A(2^{k-2}) + 2 \\ &= A(2^{k-3}) + 3 \end{aligned}$$

$$= A(2^{k-i}) + i$$

$$= A(2^{k-k}) + k$$

$$= \underbrace{A(2^0)}_0 + k$$

$$= k$$

$$n = 2^k \Rightarrow k = \log_2 n$$

$$A(n) = \log_2 n \in \Theta(\log n)$$



Binary Search AlgorithmInput : $A[0, \dots, n-1]$

Output : Index or -1

Pseudo Code:

```

 $l \leftarrow 0, r \leftarrow n-1$ 
while ( $L \leq r$ ) do
     $m \leftarrow \lfloor (l+r)/2 \rfloor$ 
    if  $K = A[m]$  return  $m$ ;
    else if  $K < A[m]$   $r \leftarrow m-1$ 
    else  $l \leftarrow m+1$ 
return -1;

```

index	0	1	2	3	4	5	6	7	8	9	10	11	12
value	3	14	27	31	39	42	55	70	74	81	85	93	98
iteration1	l						m						r
iteration2								l		m			r
iteration3								l	r				

Search value = 70

$$C_w(n) = C_w(\lfloor n/2 \rfloor) + 1, \text{ for } n > 1$$

Initial Condition

$$C_w(1) = 1$$

$$n = 2^k$$

Backward Substitution

$$C_w(2^k) = k+1 = \log_2 n + 1$$

(14)

Verify by Substitution that function $C_w(n) = \lfloor \log_2 n \rfloor + 1$ indeed satisfies equation for any positive even number

$$n = 2i, \quad i > 0$$

$$\begin{aligned} C_w(n) &= \lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 2i \rfloor + 1 = \lfloor \log_2 2 + \log_2 i \rfloor + 1 \\ &= (1 + \log_2 i) + 1 = \log_2 i + 2 \end{aligned}$$

$$\begin{aligned} C_w(\lfloor n/2 \rfloor) + 1 &= C_w(\lfloor 2i/2 \rfloor) + 1 = C_w(i) + 1 \\ &= (\lfloor \log_2 i \rfloor + 1) + 1 = \lfloor \log_2 i \rfloor + 2 \end{aligned}$$

The worst-case efficiency of Binary Search $\Theta(\log n)$

$$C_{avg}(n) \approx \log_2 n \longrightarrow C_{avg}^{yes}(n) \approx \log_2 n - 1$$

$$C_{avg}^{no}(n) \approx \log_2 (n+1)$$

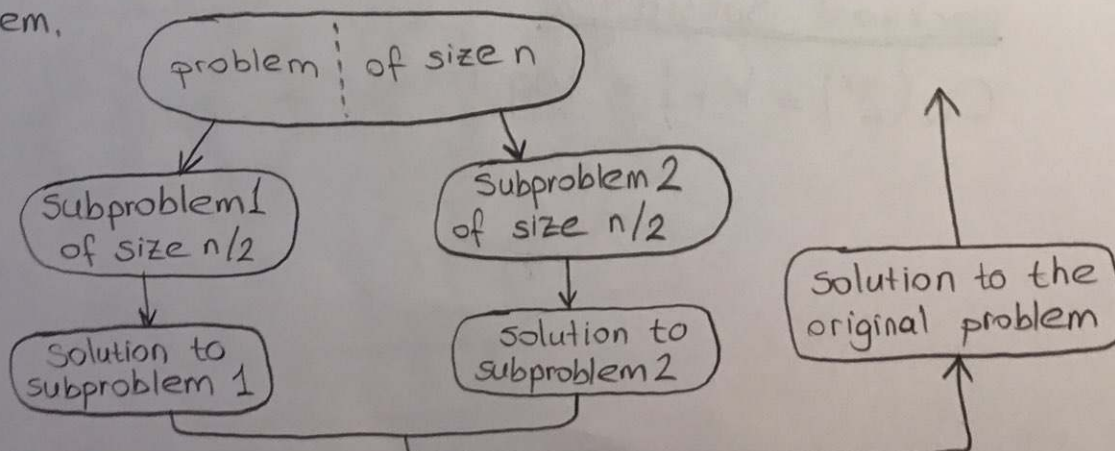
$$\lfloor \log_2 10^3 \rfloor + 1 = 10$$

$$\lfloor \log_2 10^6 \rfloor + 1 = 20$$

Divide and Conquer Algorithms

They work according to the following general plan

1. A problem's instance is divided into several smaller instances of the same problem, ideally about same size.
2. The smaller instances are solved (typically recursively though sometimes a different algorithm is employed when instances become small enough.)
3. If necessary, the solutions obtained for the smaller instances are combined to get a solution to the original problem.



Computing the sum of n

$$a_0 + a_1 + \dots + a_{n-1} = (a_0 + \dots + a_{\lfloor n/2 \rfloor - 1}) + (a_{\lfloor n/2 \rfloor} + \dots + a_{n-1})$$

Instance size : n

Subproblems : b

the subproblems that need to be solved : a [General Divide-and-Conquer Recurrence]

$$T(n) = a T(n/b) + f(n) \rightarrow \text{Running Time}$$

$f(n)$ is a function that accounts for the time spent on dividing the problem into smaller ones and combining their solutions

MASTER THEOREM

If $f(n) \in \Theta(n^d)$ where $d \geq 0$

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

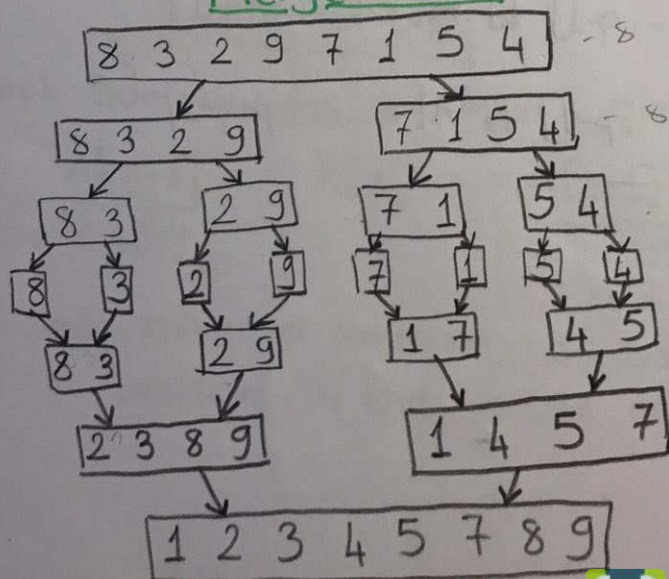
Example: $A(n) = 2A(n/2) + 1$ if $n > 1$

$$\begin{aligned} a &= 2 \\ b &= 2 \\ d &= 0 \end{aligned}$$

n 'e bağlı değil

$$\rightarrow A(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$$

Merge Sort



$$N^{\log_2 N}$$



(16)

Pseudo Code

Mergesort($A[0, \dots, n-1]$)

if ($n > 1$)

Copy $A[0 \dots \lfloor n/2 \rfloor - 1]$ to $B[0 \dots \lfloor n/2 \rfloor - 1]$

Copy $A[\lfloor n/2 \rfloor \dots n-1]$ to $C[0 \dots \lfloor n/2 \rfloor - 1]$

Mergesort($B[0 \dots \lfloor n/2 \rfloor - 1]$)

Mergesort($C[0 \dots \lfloor n/2 \rfloor - 1]$)

Merge(B, C, A)

$$C(n) = 2C(n/2) + C_{\text{merge}}(n) \quad n > 1,$$

- 4. hafta / 18.10.2018

Initial Condition

$$C(1) = 0$$

Merge($B[0, \dots, p-1], C[0, \dots, q-1], A[0, \dots, p+q-1]$)

$i \leftarrow 0, j \leftarrow 0, k \leftarrow 0$

while $i < p$ and $j < q$ do

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i], i \leftarrow i+1$

else

$A[k] \leftarrow C[j], j \leftarrow j+1$

$k \leftarrow k+1$

if $i = p$

Copy $C[j, \dots, q-1]$ to $A[k, \dots, p+q-1]$

else

Copy $B[i, \dots, p-1]$ to $A[k, \dots, p+q-1]$



- Recurrence Relation -

$$C(n) = 2C(n/2) + C_{\text{merge}}(n) \quad \text{for } n > 1$$

$$C_{\text{merge}}(n) = n - 1$$

$$\text{Initial Condition: } C(1) = 0$$

$$C_{\text{worst}}(n) = 2C_{\text{worst}}(n/2) + n - 1$$

- Backward Substitution -

$$C(n/2) = 2C(n/4) + n/2$$

$$C_{\text{worst}}(n) = 2[2C(n/4) + n/2] + n$$

$$= 4C(n/4) + n + n$$

$$= 4C(n/4) + 2n$$

$$= 8C(n/8) + 3n$$

$$= 2^i C(n/2^i) + i * n$$

$$= nC(1) + \log_2 n * n$$

$$= \boxed{n * \log_2 n}$$

$$\begin{aligned} n &= 2^i \\ i &= \log_2 n \end{aligned}$$

space complexity'si
diğerlerine göre çok
yüksek.

$$C_{\text{worst}}(n) \in \Theta(n \log n)$$

Principal Shortcoming: Linear amount of extra storage.

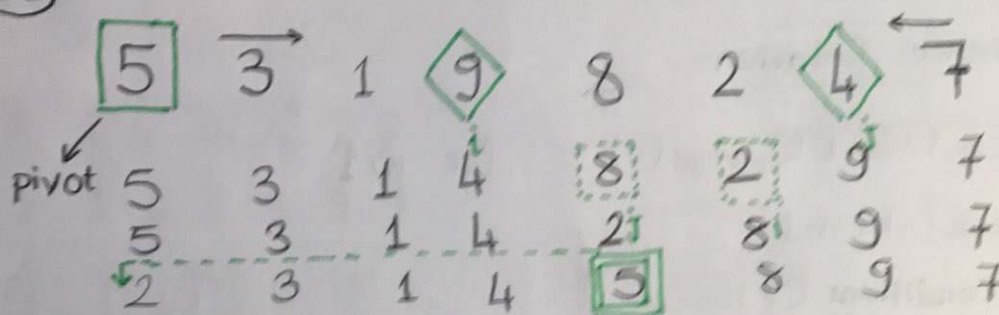
Quick Sort

Quick Sort divides the input's elements according to their value.

$$\underbrace{A[0] \quad A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \quad A[n-1]}_{\text{all are } > A[s]}$$

Obviously after a partition has been achieved, $A[s]$ will be in its final position in the sorted array

(18)



Algorithms:

(a short example
whole example on p.19)

if $l < r$

$S \leftarrow \text{Partition}(A[l, \dots, r])$

$\text{Quicksort}(A[l, \dots, S-1])$

$\text{Quicksort}(A[S+1, \dots, r])$

① Steps on encountering the first element greater than OR equal to the pivot : left-to-right scan

② Steps on encountering the first element smaller than OR equal to the pivot : right-to-left scan

Situations:

① If scanning indices i and j have not crossed e.g. $i < j$ we simply exchange $A[i]$ and $A[j]$ and resume the scans by incrementing i and decrementing j repeatedly.

② If the scanning indices have crossed over i.e. $i > j$ we have partitioned the array after exchanging the pivot with $A[j]$

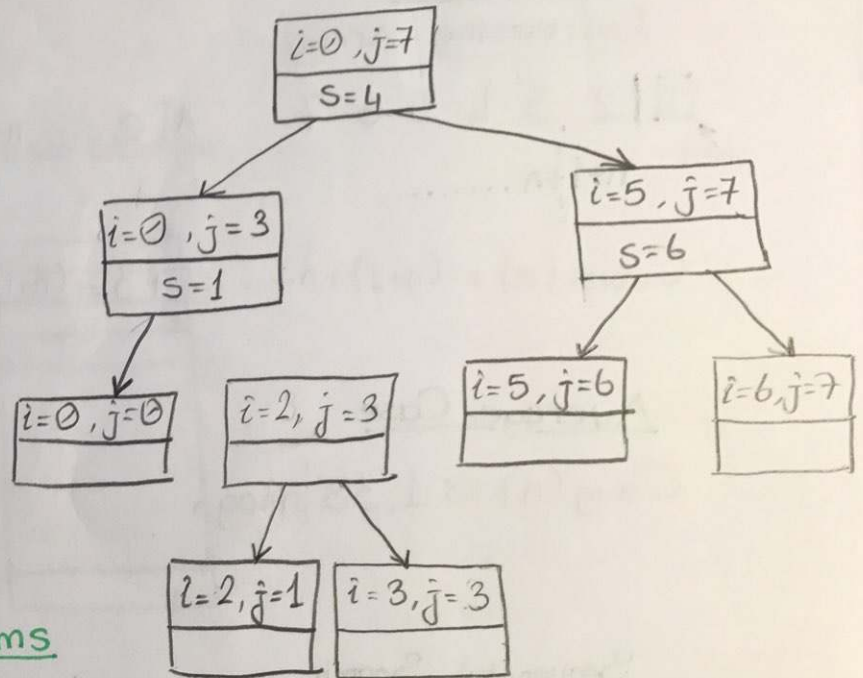
③ If the searching indices stop while pointing to the same element i.e. $i = j$ the value they are pointing to must be equal

Combine 2 and 3

Exchange $A[j]$ with pivot whenever $i > j$



0	1	2	3	4	5	6	7
5	3	1	9	8	2	4	7
5	3	1	4	8	2	9	7
5	3	1	4	2	8	9	7
2	3	1	4	5	8	9	7
2	1	3	4	5	8	7	9
1	2	3	4	5	7	8	9



Partition Algorithms

$p \leftarrow A[l]$
 $i \leftarrow l, j \leftarrow r+1$
 repeat
 repeat $i \leftarrow i+1$ until $A[i] \geq p$
 repeat $j \leftarrow j-1$ until $A[j] \leq p$
 swap $(A[i], A[j])$

until $i \geq j$
 swap $(A[i], A[j])$ //undo last swap when $i \geq j$
 swap $(A[l], A[j])$

Best Case

If all the splits happen in the middle of corresponding subarrays, we will have the best case.

~Recurrence Relation~

$$C_{\text{best}}(n) = 2C_{\text{best}}(n/2) + n$$

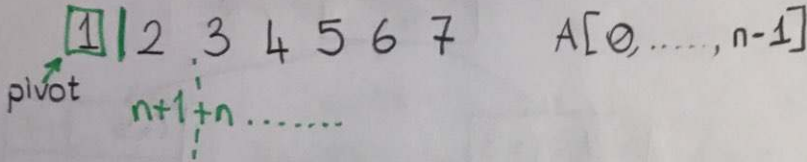
$$C_{\text{best}}(1) = 0$$

$$C_{\text{best}}(n) = \Theta(n \log n)$$

(20)

Worst-Case

Increasing Array

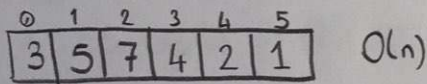


$$C_{\text{worst}}(n) = (n+1) + n + \dots + 3 = \frac{(n+1) \cdot (n+2)}{2} - 3 \Rightarrow \Theta(n^2)$$

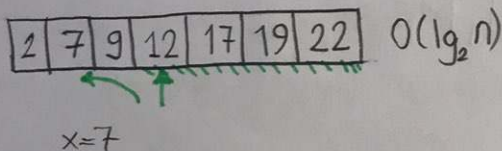
Average Case

$$C_{\text{avg}}(n) \approx 1,38 n \log n$$

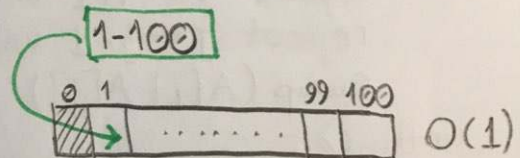
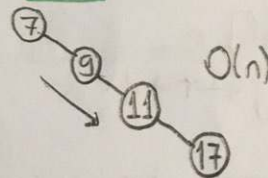
Sequential Search



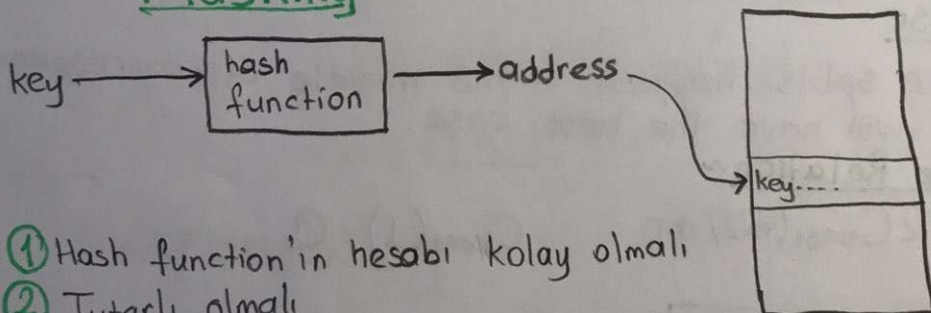
Binary Search



BST



Hashing



- ① Hash function'in hesabı kolay olmalı
- ② Tutarlı olmalı
- ③ Adresin olabildiğince uniform dağılımı sağlanmalı

key mod m ← tablo uzunluğu

$m = 10 \rightarrow 2^x$

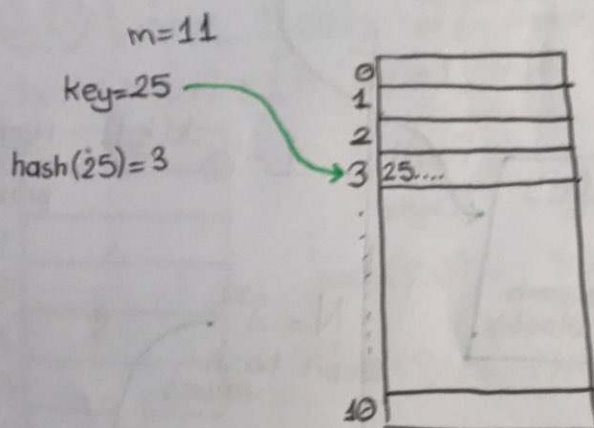
key → 1000, 10, 40, 50

★ m 'in asal sayı olması performansı artırır



① Divison Method

$$\text{hash}(\text{key}) = \text{key} \bmod m \quad \rightarrow \text{tablo uzunlu\u011fu}$$

② Multiplication Method

A: constant $0 < A < 1$

$$A = \frac{\sqrt{5}-1}{2} = 0,618033$$

$$\text{hash}(\text{key}) = \lfloor m * [(\text{key} * A) \bmod 1] \rfloor$$

$$(\text{key} * A) \bmod 1 = \text{key} * A - \lfloor \text{key} * A \rfloor$$

tablo
uzunlu\u011fu

$m=10.000$ $k=123456$

$$\text{hash}(123456) = \lfloor 10.000 * (123456 * 0,618033) \bmod 1 \rfloor$$

$$\underbrace{763.000,004151}$$

$$= \lfloor 10.000 * 0,004151 \rfloor$$

$$= \lfloor 41,51 \rfloor$$

$$\text{hash}(123456) = 41$$

address

(22)

Birthday Paradox

$$365 \rightarrow \frac{365-1}{365} * \frac{365-2}{365} * \dots * \frac{365-(k-1)}{365}$$

3 kişinin doğum günü farklı

table size: m

$$\frac{m-1}{m} * \frac{m-2}{m} * \dots * \frac{m-(k-1)}{m} \sim e^{-\frac{k(k-1)}{m}}$$

m tablo uzunluğunda çakışmama olasılığı

$$1 - e^{-\frac{k(k-1)}{m}} : \text{Hash collision probability}$$

$$N = 2^{32}$$

32-bit hash values

after 250.000 insert %100

77163 insert %50

Key

- ① Integer
- ② Floating number
- ③ String

ikisi farklı değer döndürmeli
"ELMA"
"MALE"

Horner's Method:

$$\text{key} = R^{L-1} * \text{str}[0] + \dots + R^0 * \text{str}[L-1]$$

31 (olarak seçilir) length of string Büyük harf ise: $\text{str}[0] - 'A' + 1$

④ Compound key

31.08.1997

$$\rightarrow 31 * R + 8 * R^2 + 1997 * R^3$$

31



Çakışma Problemine Çözüm Önerileri

1] Open Addressing

table size $m \sim 2 * N$ olduğunda rahat

delete işlemi sıkıntılı

(ya yeniden düzenlenmeli ya da aramaya devam etmesi için uygun flag konur)

1.1] Linear Probing

$$\text{hash}(\text{key}, i) = [h'(\text{key}) + i] \bmod m$$

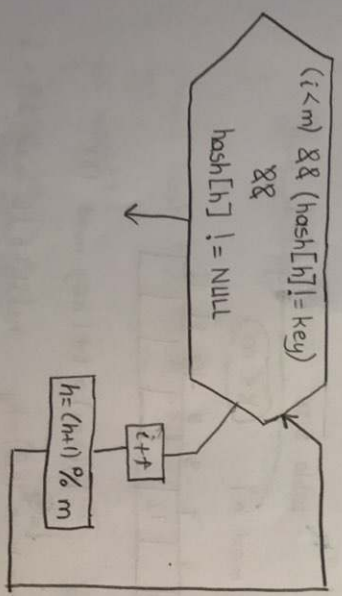
$i = 0, \dots, m-1$

$m = 7$
 $\text{key} = 9$

i
 $\text{hash}(9, 0) = 9 \bmod 7 = 2$
 $i+1$
 $\text{hash}(9, 1) = (9+1) \bmod 7 = 3$
 $i+2$
 $\text{hash}(9, 2) = (9+2) \bmod 7 = 4$

$\text{key} = 13$
 $\text{hash}(13, 0) = 6$
 $\text{hash}(13, 1) = (13+1) \bmod 7 = 9$

key
 $i = 0$
 $h = \text{key} \bmod m$



1.2 Quadratic Probing

$$h(k, i) = (h^1(k) + i^2) \bmod m$$

Linear Probing'de her yenileşecek eleman için
ölmüşün 1'er pozisyon kayma durumu olabilir.
Bunu önlemek için arada boşluk bırakarak
atılma yapabilecek bir çözüm sunar.

0	
1	
2	2
3	3
4	
5	
6	9

$$\text{hash}(9, 0) = 9 \bmod 7 = 2$$

$$\text{hash}(9, 1) = (2+1) \bmod 7 = 3$$

$$\text{hash}(9, 2) = (2+2^2) \bmod 7 = 6$$

$$h(k, i) = (h^1(k) + C_1 * i + C_2 * i^2)$$

$$C_1 = 2 \quad C_2 = 1$$

$$\text{hash}(9, 1) = (2 + 2 * 1 + 1 * 1) \bmod 7 = 5$$

$$C_1 = 2 \quad i = 1 \quad C_2 = 1$$

1.3 Double Hashing

$$\text{hash}(key, i) = [h_1(key) + i * h_2(key)] \bmod m$$

→ adım büyüklüğü

$$h_1(key) = key \bmod m$$

table size

$$h_2(key) = 1 + (key \bmod k)$$

$$(k < m)$$

→ genelde asal

0	1	2	3	4	5	6	7	8	9	10	11	12
	+9			69	58		72					

$$h_1(key) = key \bmod 13$$

$$h_2(key) = 1 + (key \bmod 11)$$

$$\text{Ex: } \text{hash}(14) = ?$$

$$i=0 \rightarrow \text{hash}(14, 0) = h_1$$

$$h_1(14) = 14 \bmod 13 = 1$$

$$i=1$$

$$\text{hash}(14, 1) = 1 + [1 + 14 \bmod 11] * 1 = 1 + 4 = 5$$

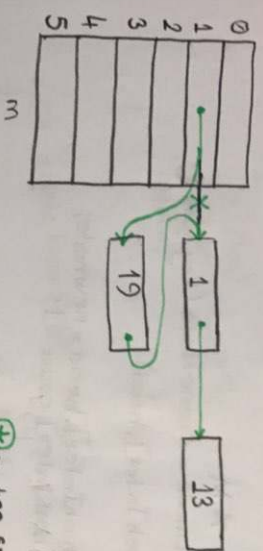
$$i=2$$

$$\text{hash}(14, 2) = [1 + 4 * 2] \bmod m = 9$$

$$h_1(key)$$

$$h_2(key)$$

[2] Separate Chaining



key = 13 % 6 → 1
 key = 1 % 6 → 1
 key = 19 % 6 → 1

* m too small → chains too long
 * Statistical optimum value → $M \sim N/5$

$M < N$
 maximum → $M = N$

```
struct node
{
```

```
    int key;
    char name[40];
    struct node *next;
};
```

```
struct hash
{
```

```
    struct node *head;
    int count;
}
```

```
struct hash *hashTable = NULL;
```

```
int main()
```

```
{
    int key;
    char name[40];
```

```
    hashTable = (struct hash*) calloc (M, sizeof (struct hash));
    if (hashTable == NULL)
```

```
{
```

```
    *insertToHash (key, name);
```

```
    *searchInHash (key);
```

```
    *deleteFromHash (key);
```

```
return 0;
```

(26)

```

void inserttoHash (int key, char name[])
{
    int hashIndex;
    hashIndex = key % M;
    struct node *newnode = createNode (key, name);
    if (!hashTable[hashIndex].head)
    {
        hashTable[hashIndex].head = newnode;
        hashTable[hashIndex].count = 1;
        return;
    }
    newnode->next = hashTable[hashIndex].head;
    hashTable[hashIndex].head = newnode;
    hashTable[hashIndex].count++;
    return;
}

struct node *createNode (int key, char *name)
{
    struct node *newnode;
    newnode = (struct node*) malloc (sizeof(struct node));
    newnode->key = key;
    strcpy (newnode->name, name);
    newnode->next = NULL;
    return newnode;
}

void searchInHash (int key)
{
    int hashIndex = key % M, flag = 0;
    struct node *myNode;
    myNode = hashTable[hashIndex].head;
    if (!myNode)
    {
        printf("Search element unavailable in hash table\n");
        return;
    }
    while (myNode != NULL && flag == 0)
    {
        if (myNode->key == key)
        {
            printf("Voter ID : %d\n", myNode->key);
            printf("Name : %s\n", myNode->name);
            flag = 1;
        }
    }
}

```




```

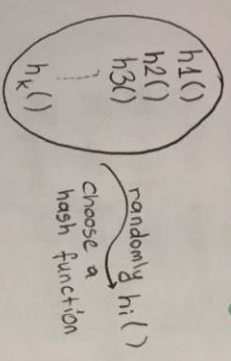
myNode = myNode -> next;
}
if (!flag)
    printf("yok!");
return;
}

void deleteFromHash (int key)
{
    int hashIndex = key % M;
    struct node *temp, *myNode;
    myNode = hashTable[hashIndex].head;
    if (!myNode)
    {
        printf("Given data is not present in hash Table!\n");
        return;
    }
    temp = myNode;
    while (myNode != NULL && flag != 1)
    {
        if (myNode -> key == key)
        {
            flag = 1;
            if (myNode == hashTable[hashIndex].head)
                hashTable[hashIndex].head = myNode -> next;
            else
                temp -> next = myNode -> next;
            hashTable[hashIndex].count--;
            free(myNode);
        }
        temp = myNode;
        myNode = myNode -> next;
    }
    if (flag)
    {
        printf("Data deleted successfully from Hash Table!\n");
    }
    else
        printf("Given data is not presented in hash Table!\n");
}

```

28

3 Universal Hashing



15, 22, 29 $\xrightarrow{7}$ hash $\rightarrow 1$

- Kullman Hash Funksiyonları -

$$h_{a,b}(key) = [(a * k + b) \bmod p] \bmod m$$

$a, b, p \rightarrow$ chosen randomly

$$p = 17, m = 6, a = 3, b = 4$$

$\rightarrow$ Table size is not random

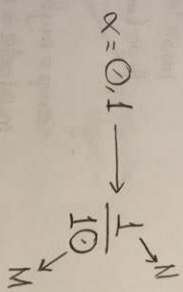
$$h(8) = ((3 * 8 + 4) \bmod 17) \bmod 6 = 5$$

open addressing : $M > N$
 separate chaining : $M < N$

$$\alpha = \frac{N}{M} : \text{load factor}$$

open addressing linear probe

α	# of probe
0,1 $\rightarrow$	1, 11
0,2 $\rightarrow$	1, 28
0,3 $\rightarrow$	1, 52
0,4 $\rightarrow$	1, 89
0,5 $\rightarrow$	2, 5
0,6 $\rightarrow$	3, 62
0,8 $\rightarrow$	13, 00
0,9 $\rightarrow$	50, 50



$$\left(\frac{1}{2}\right) \checkmark$$



Unsuccessful Search

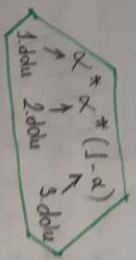
$P(\text{first location is occupied}) = \alpha$

$P(\text{empty cell}) = (1 - \alpha)$

$P(\text{first two is occupied}) = \alpha^2$

$P(\text{probe terminates 2 steps}) = \alpha^2 * (1 - \alpha)$

$P(\text{probe terminates } k \text{ steps}) = \alpha^{k-1} * (1 - \alpha)$



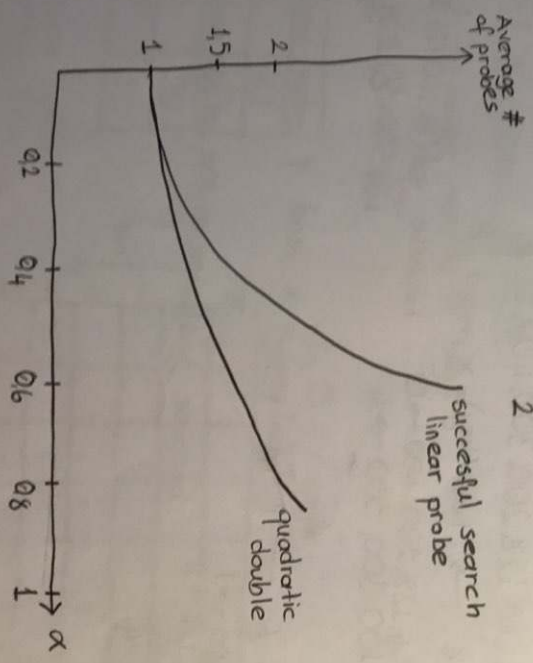
Average # of steps:

$$\sum_{k=1}^{\infty} k * \alpha^{k-1} * (1 - \alpha)$$

$\Rightarrow$ geometric series = $\frac{1}{1 - \alpha}$

Expected number of probes in unsuccessful search.

insertion
expected # of probes $\rightarrow \frac{1 + \frac{1}{1 - \alpha}}{2}$

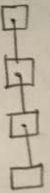


30

Separate Chaining

① Unsuccessful Search

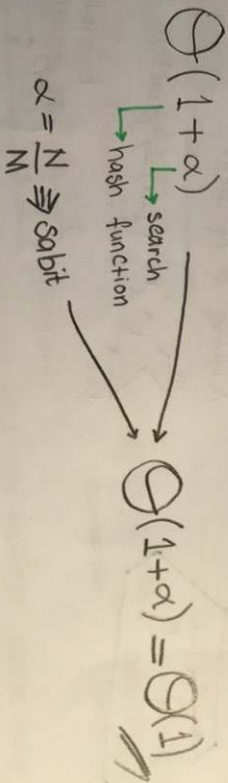
$$\alpha = \frac{N}{M}$$



Worst case: Butin elementar ayni adreste.

average case : $\alpha = \frac{N}{M}$

Expected length of a probe $\Rightarrow \alpha = \frac{N}{M}$



keys : 64, 100, 128, 200, 300, 400, 500

table size = 8 $\rightarrow$ hash(key) = x mod 8

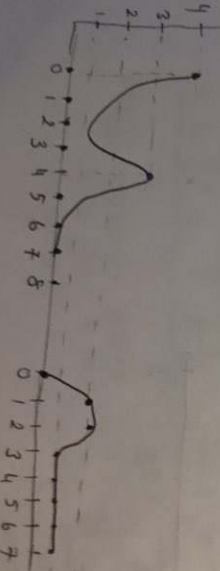
64, 128, 200, 400 $\rightarrow$ 0 because gcd(64, 100, 128, 200, 300, 400, 500, 8) = 4

100, 300, 500 $\rightarrow$ 4

table size = 7 $\rightarrow$ hash(key) = x mod 7

64, 400 $\rightarrow$ 1
100, 128 $\rightarrow$ 2
500 $\rightarrow$ 3
200 $\rightarrow$ 4
300 $\rightarrow$ 5

because gcd(64, 100, 128, 200, 300, 400, 500, 7) = 1/7



Dynamic Programming

→ Planning Richard Bellman (1950's)

Example Fibonacci Numbers

0 1 1 2 3 5

int fibonacci(int n)

{

if (n == 0)

return 0;

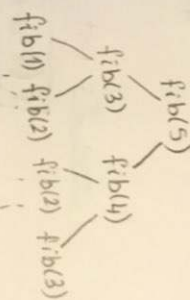
if (n == 1)

return 1;

return fibonacci(n-1) + fibonacci(n-2);

}

→ n=0 : fibonacci(n)=0
n=1 : fibonacci(n)=1
n>1 : fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)



Recurrence Relation:

fibonacci(n) = fibonacci(n-1) + fibonacci(n-2) (1, 6)ⁿ

Summation

← çok fazla tekrarlanon hesaplan

0	1	2	3	4	5	6	7	8
0	1	1	2	3	5			

fibonacci

* hesaplamaları tek tek yeniden değil, bu dizideki gözde daha önce hesaplanmış değerleri kullanıyor.

Example 0-1 Knapsack Problem

abandon select

W → weight capacity of bag

Brute force: 2ⁿ

Ex:

item	weight	value
1	2	12
2	1	10
3	3	20
4	2	15

P(i,j) = maximum profit from item 1 to i from weight 1 to j

	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

maximize $\sum v_i$ subject to W

maximum kazanç

(32)

Case 1: thief takes item i

$$P(i, w) = P(i-1, w-w_i) + v_i$$

$$i=3, w_i=2$$

$$P(3, 4) = P(2, 2) + v_3$$

4 kılığa gelindiğinde
maximum kazanç

$$P(3, 3) = P(2, 1) + v_3$$

esya 3 kılığa kadar
kazanç

Case 2: not takes item i

$$P(i, w) = P(i-1, w)$$

$$P(i, w) = \begin{cases} P(i-1, w) & \text{if } w_i > w \\ \max \{ v_i + P(i-1, w-w_i), P(i-1, w) \} & \text{others} \end{cases}$$

initialization $\rightarrow$

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	10	12	12
2	0	0	10	22	22
3	0	0	10	22	32
4	0	10	15	25	30

weight $\rightarrow$

$w_1 = 2$

$w_2 = 1$

maximum kazanç

$$P(i, 0) = 0, P(i, 0) = 0$$

$$P(1, 1) = 0, P(1, 1) = 0$$

$$P(1, 2) = \max \{ 10 + 0, 0 \} = 10$$

$$P(0, 1) = 10, P(0, 1) = 10$$

$$P(1, 3) = \max \{ P(0, 1) + 10, P(0, 3) \} = 10$$

$$P(2, 1) = \max \{ P(1, 0) + 10, P(1, 1) \}$$

$$P(2, 2) = \max \{ P(1, 2), P(1, 1) + 10 \}$$

$$P(2, 3) = \max \{ P(1, 2) + 10, P(1, 3) \} = 22$$

$$P(2, 4) = \max \{ P(1, 3) + 10, P(1, 4) \} = 22$$

$$P(3, 1) = P(2, 1) \rightarrow 3 > 1$$

$$P(3, 2) = P(2, 2) \rightarrow 3 > 2$$

$$P(3, 3) = \max \{ P(2, 0) + 20, P(2, 3) \} = 22$$

$$P(3, 4) = \max \{ P(2, 1) + 20, P(2, 4) \} = 30$$

$$P(3, 5) = \max \{ P(2, 2) + 20, P(2, 5) \} = 32$$

$$w_4 = 2$$

$$P(4, 1) = P(3, 1) = 2 > 1$$

$$P(4, 2) = \max \{ P(3, 0) + 15, P(3, 2) \} = 15$$

$$P(4, 3) = \max \{ P(3, 1) + 15, P(3, 3) \} = 25$$

$$P(4, 4) = \max \{ P(3, 2) + 15, P(3, 4) \} = 30$$

$$P(4, 5) = \max \{ P(3, 3) + 15, P(3, 5) \} = 34$$

$$P(4, 5) = \max \{ P(3, 3) + 15, P(3, 5) \} = 34$$



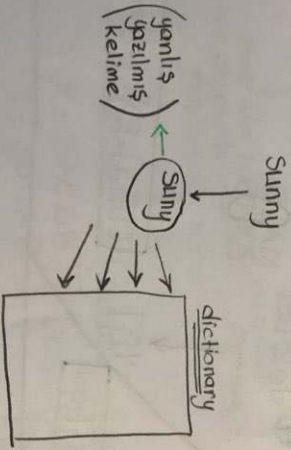
0	1	2	3	4	5
0	0	0	0	0	0
1	0	0	12	12	12
2	0	10	12	22	22
3	0	10	12	22	30
4	0	10	15	25	30

Reconstructing Optimal Solution

$w_i=2$ not taken $\uparrow$ $p[i,j] = p[i-1,j]$
 $w_i=1$ taken $\leftarrow$ $p[i,j] \rightarrow p[i-1,j-w_i]$
 $w_i=3$ taken $\leftarrow$ $p[i,j] \rightarrow p[i-1,j-w_i]$
 $w_i=2$ taken $\leftarrow$ $p[i,j] \rightarrow p[i-1,j-w_i]$

Hangi item'ler alınmış?

Edit Distance

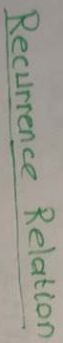


The Levenshtein Algorithm (Edit Distance)

sunny $\rightarrow$ sunny
 sunny $\rightarrow$ sun-y
 sunny $\rightarrow$ Sunny
 Yapılabilecek olası hatalar

dist (MATHS, ARTS)

MATHS
 ART-S
 MATHS
 ARTS

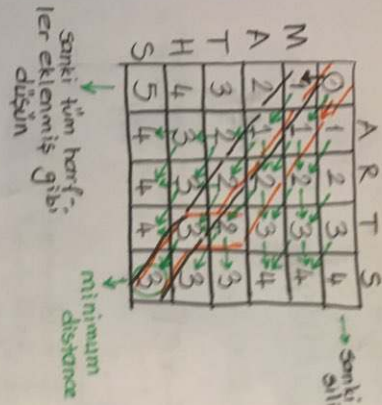
$$X[1, \dots, m] \quad \forall [1, \dots, n]$$


Recurrence

$$ED[i, j] = \begin{cases} ED[i-1, j-1] + 0 & : \text{copy} \\ ED[i-1, j-1] + \text{cost} & : \text{change} \\ \min \begin{cases} ED[i-1, j] + \text{cost} & : \text{insert} \\ ED[i, j-1] + \text{cost} & : \text{delete} \end{cases} & \text{otherwise} \end{cases}$$

if ($x[i] = y[j]$)

Initialize: $ED[i,0] = i$, $ED[0,j] = j$



MATHS ne oldu da ARTS oldu
Sorusunun cevabı bulunulabilir

1. deneme:

MATHS
ARTS

2. deneme:

MATHS
ARTS

Longest Common Subsequence

X: BACDB } LCS: BCB
Y: BDCAB } GDB
BAB

$$LCS[i,j] = \begin{cases} LCS[i-1,j-1] + 1, & \text{if } x[i] = y[j] \\ \max \{ LCS[i-1,j], LCS[i,j-1] \}, & \text{otherwise} \end{cases}$$

8. Hafta

<A,C,G,T>

X: ATCTGAT
Y: TGCATA

Longest Common Subsequence of X and Y

$LCS(X,Y) = TCTA$
 $LCS(X,Y) = TGAT$

Subsequence

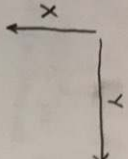
KOZALAK

KAL
KOL
KOLA
KALK

36

$X = \langle x_1, x_2, \dots, x_m \rangle$
 $Y = \langle y_1, y_2, \dots, y_n \rangle$ } LCS

$S = \langle s_1, s_2, \dots, s_k \rangle$
 $s_{i_1}, s_{i_2}, \dots, s_{i_k}$
 $i_1 < i_2 < \dots < i_k$



Örnek: $X: \langle BACDB \rangle$
 $Y: \langle BDCB \rangle$

Hangi harfleri aldığımızı dair

		B	A	C	D
B	0	1	0	0	0
D	0	0	1	0	1
C	0	0	0	1	0
B	0	1	0	0	1

Hangi harfleri aldığımızı dair

Initial:

if $i = 0 \rightarrow C[i, j] = 0$
 if $j = 0 \rightarrow C[i, 0] = 0$

$C[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ \max \{ C[i-1, j-1] + 1, C[i-1, j], C[i, j-1] \} & \text{otherwise} \end{cases}$

Hangi harfleri aldığımızı dair
 $[M, N]$ gözünden ilerlemeye başladığımızda
 harf ortak olarak alındı mı alınmadı mı
 belli olmayan karmışa oluyor.

ÇÖZÜM

Yeni bir Matris

$B[6, 5]$

ADD: 0
 SKIPX: 1
 SKIPPY: 2

$x_i = y_j \rightarrow B[i, j] = ADD$
 $x_i \neq y_j \rightarrow C[i-1, j] > C[i, j-1]$ then

$x[i]$ not in LCS
 $B[i, j] = SKIPX$
 else

$B[i, j] = SKIPPY$
 $y[j]$ is not in LCS

$B[6, 5]$

BCB



Longest Common Subsequence

```

while ((i=0) && (j=0))
{
    switch (b[i][j])
    {
        case ADD:
            add X[i] to front of LCS
            i--;
            j--;
            break;
        case SKIPX:
            i--;
            break;
        case SKIPY:
            j--;
            break;
    }
}
}

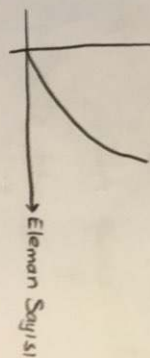
```


(38)

- 1. Vize Cevapları -

5/1 Çalışma Zamanı : $\cancel{8}N^2 + 13N^{4/3} + 12N + 3$ $N \rightarrow 1$ Saat $2N \rightarrow ?$ $\downarrow$ Saat(Bellek ve çalışma zamanının hızı $\rightarrow \times 2$)
Çalışma süresi

Örnek: 2 saat



5/2

$$3n^3 + 20n^2 + 5 \rightarrow O(n^3)$$

 $\rightarrow$ olduğu gösteriniz.

$$3n^3 + 20n^2 + 5 \rightarrow O(n^3) \rightarrow \text{BÖYLE SAĞMALARMIŞ}$$

$$f(n) \leq c \cdot g(n)$$

$$3n^3 + 20n^2 + 5 \leq c n^3 \quad n > n_0, \quad c > 0$$

$$n = 1$$

$$c = 28$$

$$3 + 20 + 5 \leq 28$$

$$28 \leq 28$$

10/3

İki algoritmanın performans karşılaştırması kistası.

Time complexity, Space complexity

 $\rightarrow$ Best Case $\rightarrow$ Average Case $\rightarrow$ Worst-Case $\rightarrow$ Theoretical Analysis $\rightarrow$ Empirical Analysis (Girdiği artırıp denemeler yapmak)

15/4

10/5

void bubbleSort(int N, int *dizi) {

int i=0, j=0, swap, tmp;

while ((i < N-2) && (swap==1)) {

swap=0;

j=0;

while ((j < N-2-i)) {

if ((j < N-2-i)) {

swap=1;

tmp=dizi[j+1];

dizi[j+1]=dizi[j];

dizi[j]=tmp;

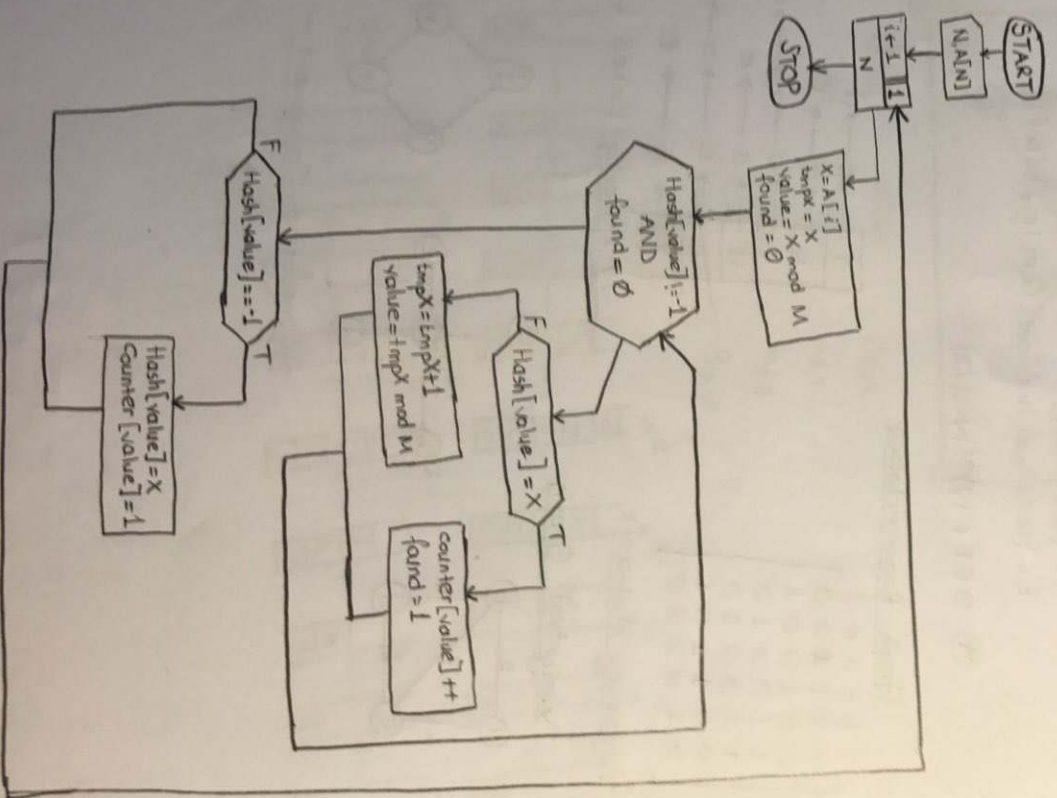
6/6

(1. ifade 2. ye ghere.)

$n(n+5)$	ve	$5000n^2$	→ E
2^{n-1}	ve	2^{n+1}	→ E
$\log(n)$	ve	$\ln(n)$	→ E
$1000n^2$	ve	$0.001n^3$	→ K
$(n-5)!$	ve	2^n	→ B
$(10n+5)\log n$	ve	$100n+(mn)\log n$	→ E

(E: exit, B: bəyqə, K: kiçik)

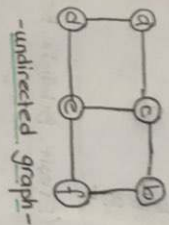
25/7



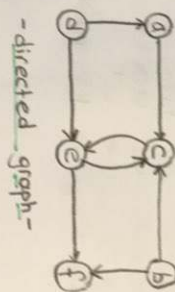
(40)

Graph $\rightarrow G = \langle V, E \rangle$

①
can be graph



-undirected graph-



-directed graph-

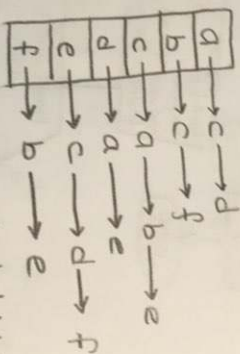
$V = \{a, b, c, d, e, f\}$
 $E = \{(a,b), (a,c), (b,c), (c,d), (c,e), (d,e), (e,f), (f,b)\}$

* $0 \leq E \leq |V| * (|V|-1)/2$

Graph Representation

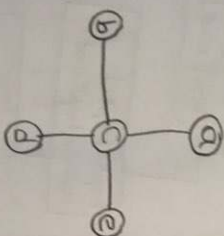
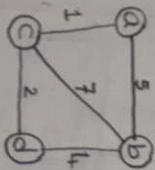
	a	b	c	d	e	f
a	0	0	1	1	0	0
b	0	0	1	0	0	1
c	1	1	1	0	0	1
d	1	0	0	0	1	0
e	0	0	1	1	0	1
f	0	1	0	0	0	0

~ Adjacency Matrix ~

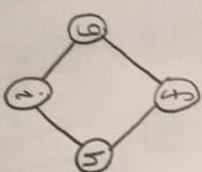


~ Adjacency Linked List ~

Weighted Graph

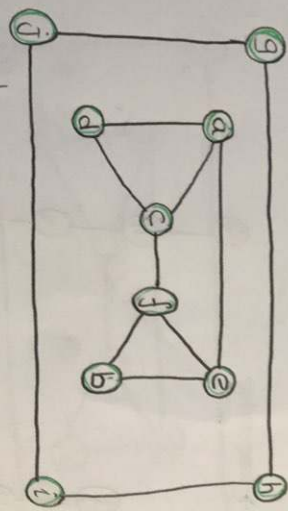


~ Connected Component ~



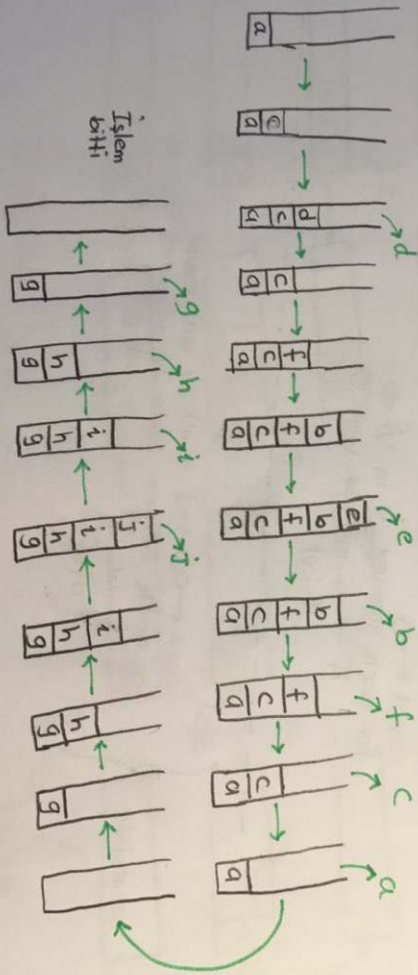
Depth-First-Search

Decrease-by-one technique



$d_{3,1}$ $e_{5,2}$ $j_{10,7}$
 $c_{2,5}$ $b_{6,3}$ $i_{9,8}$
 $a_{1,6}$ $f_{4,4}$ $h_{8,9}$
 $g_{7,10}$

LIFO
 ↓
 STACK kullanılır.
 (push & pop yapılacaktır)

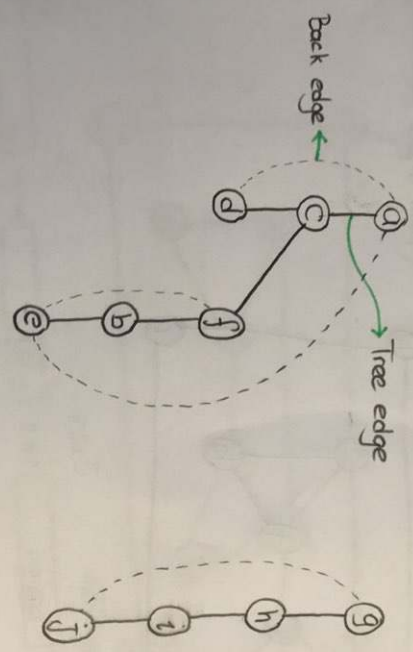


İşlem bitti

42

Depth-First Search Forest

Tree edge
Back edge



DFS(G)

mark each vertex in V with $\emptyset$ as a mark of being unvisited
 count $\leftarrow \emptyset$
 for each vertex v in V do
 if v is marked with $\emptyset$
 dfs(v)

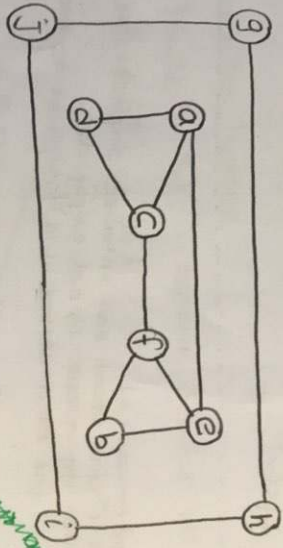
dfs(v)
 count $\leftarrow$ count + 1, mark v with count
 for each vertex w in V adjacent to v do
 if w is marked with $\emptyset$
 dfs(w)
 recursive



(43)

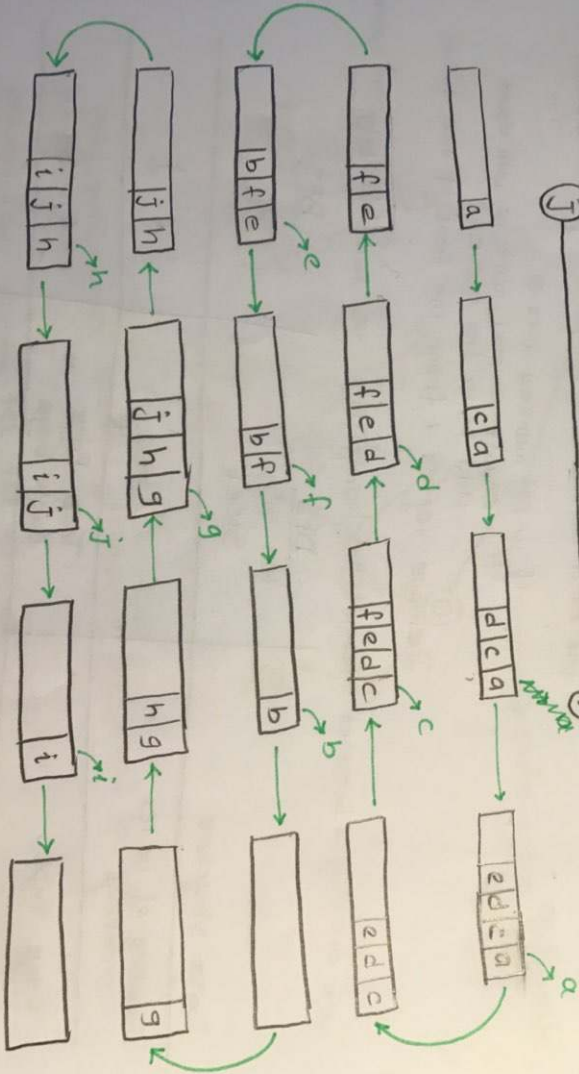
Breadth-First-Search

cesur / brave → depth
tedbirli / cautious → breadth



FIFO
↓
QUEUE kullanılır.
(queue & enqueue yapılarak)

$a_1 c_2 d_3 e_4 f_5 b_6$
 $g_7 h_8 j_9 i_{10}$



İŞLEM
TAMAM



BFS(G)

mark each vertex in V with 0 as a mark of being unvisited
 count $\leftarrow 0$
 for each vertex v in V do
 if v is marked with 0
 bfs(v)

iterative

bfs(v)
 count $\leftarrow$ count + 1
 mark v with count and initialize a queue with v
 while the queue is not empty do
 for each vertex w in V adjacent to the front's vertex v do
 if w is marked with 0
 count $\leftarrow$ count + 1, mark w with count
 add w to the queue
 remove vertex v from the front of the queue

web crawling $\rightarrow$ Internet siteside link icinde link ve burdaki bilgileri okuma

DFS

BFS

Data structure	Stack	Queue
Number of vertex orderings	2	1
Edge types	Tree Edge Back Edge	Tree Edge Cross Edge
Applications		Web crawling (belirli sinir yernek icin) Social network Map
Efficiency for adjacent matrix	$O(V^2)$	$O(V^2)$
Efficiency for adjacent linked-list	$O(V + E)$	$O(V + E)$

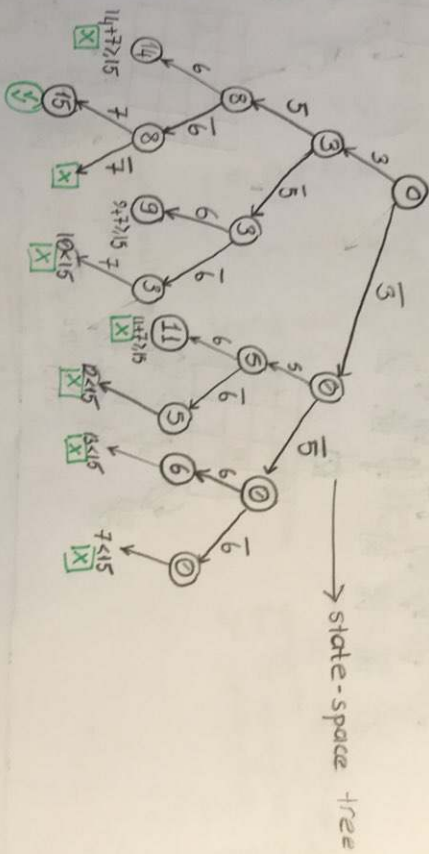
Backtracking → DFS with pruning



Sub Set Sum Problem

$S = \{s_1, s_2, \dots, s_n\}$, toplamı d olan alt kümeler? ($s_1 s_2 s_3 \dots s_n$)

Ex: $S = \{3, 5, 6, 7\}$, $d = 15$



8-vezir problemi (n-queen problem)

Satranç tahtasına birbirini görmeyecek (yemeyecek) şekilde 8 veziri yerleştirme.

1 Her bir durumu tek tek denemek

$$\binom{64}{8} = \frac{64!}{8! \cdot (64-8)!} = 4,426,165,368 \text{ deneme}$$

2 One per row

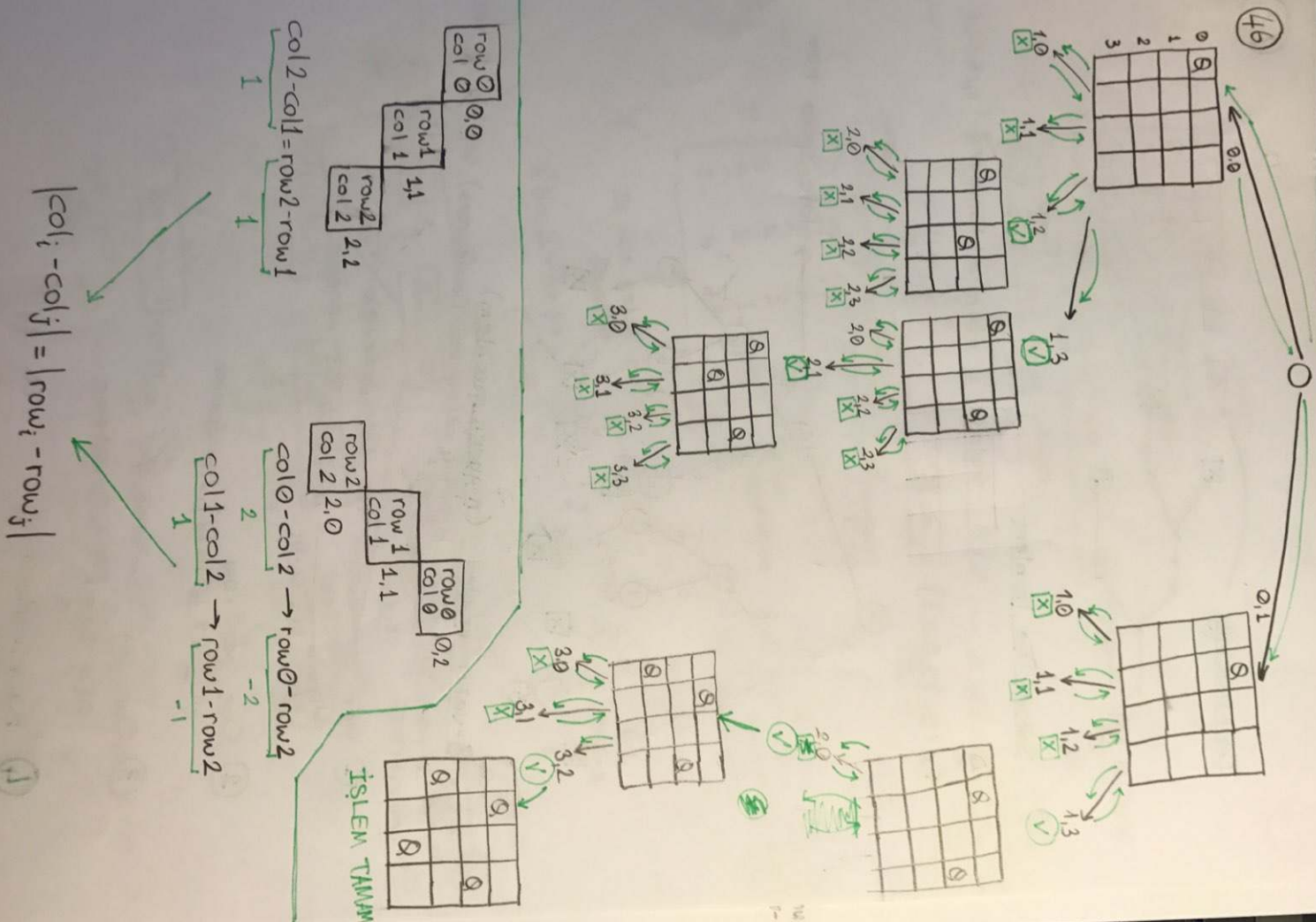
$$n^n = 8^8 = 16,777,216 \text{ deneme}$$

3 One per row and per column

$$n! = 8! = 40,320 \text{ deneme}$$

4 Backtracking

2047 deneme



1	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

→ Vezirleri yerleştirdiğim matris

* Ekstra bir diziyi hangi vezir hangi satır/sütunda onu saklarız.
 0 1 2 3
 0 1 2 3 → Hangi vezir (s. satır veziri)
 2 4 1 3 ← Hangi sütunda

C =

	J1	J2	J3	J4	
9	2	7	4	a	İş yapacak işler
6	4	3	7	b	
5	8	1	8	c	
7	6	9	4	d	

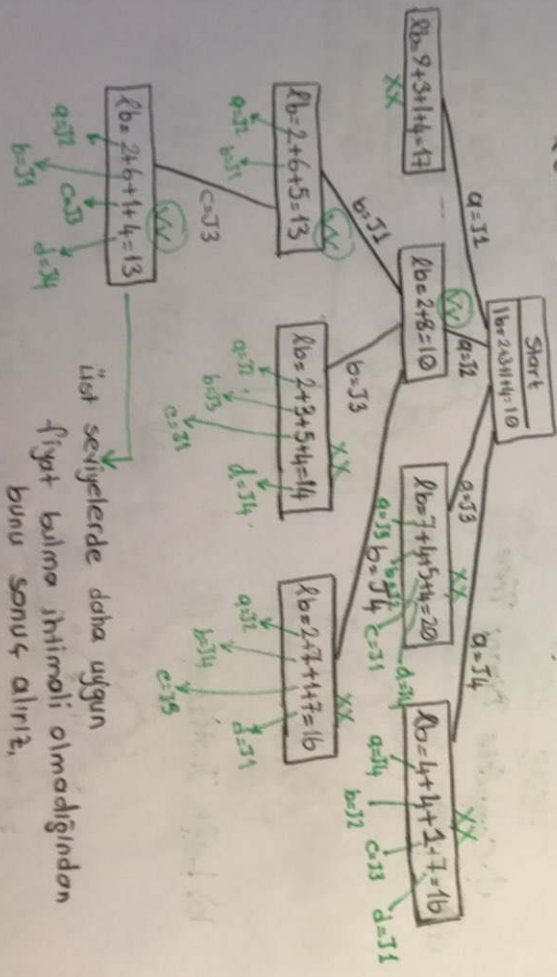
Elimizde 02 poro &e
yapılacak işler var bunu
en az maliyetle yapma-
ya çalışıyoruz.

Branch and Bound

→ BFS

İş için tekil edilen fiyatlar

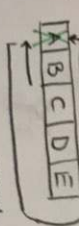
$$Qb = 2 + 3 + 1 + 4 = 10$$



Priority Queue

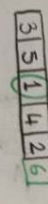
First - in - first - out

$A \leftarrow B \leftarrow C \leftarrow D$



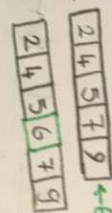
- array
- linked list

① - unsorted array -

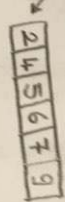


- insert $O(1)$
- remove $O(N)$

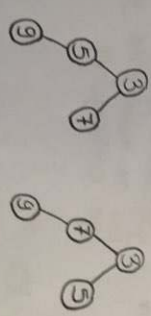
② - sorted array -



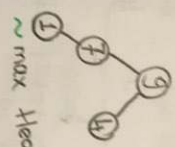
- insert $O(N)$



- remove $O(1), O(N)$



~ min heap ~

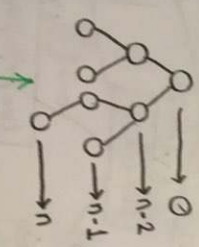
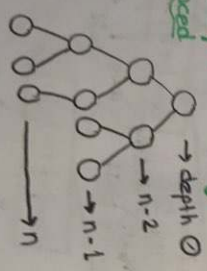


~ max heap ~

- Heap Tree -

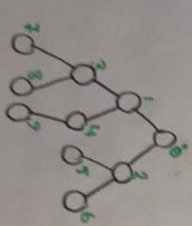
Complete Binary Tree

a) Balanced



All the nodes at depth 0 through $n-2$ have 2 children!
Balanced Tree condition

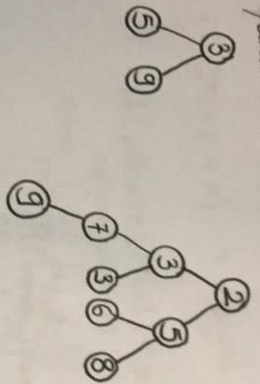
b) Left Justified



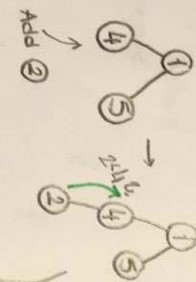
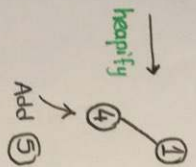
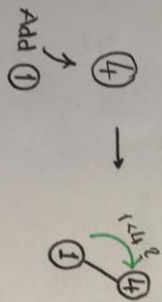
left-justified

NOT LEFT-JUSTIFIED

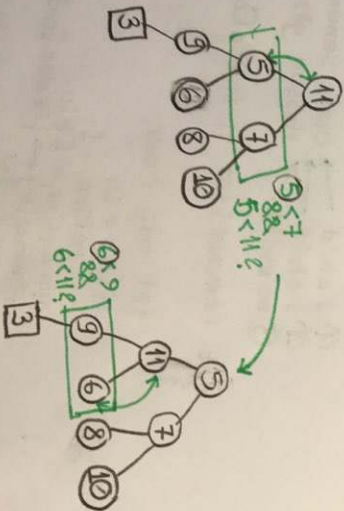
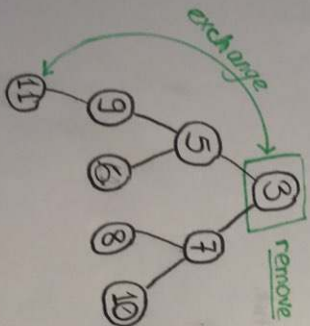
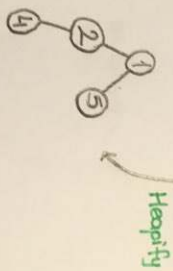
② a parent's value $\leq$ its children's values : min Heap



Heapify

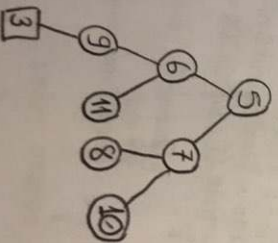


$O(\lg_2 n) \leftarrow \text{insert}$ UPHEAP



$O(\lg_2 n) \leftarrow \text{remove}$

DOWNHEAP




```
void insert(int newItem, int n, int pQueue[])
```

```
{
    int i, stop = 0, parent;
    pQueue[n] = newItem;
```

```
    i = n;
    n++;
    while ((i > 0) && (!stop))
```

```
{
    parent = (i-1)/2;
```

```
    if (pQueue[i] < pQueue[parent])
```

```
{
    pQueue[i] ↔ pQueue[parent];
    i = parent;
```

```
} else
    stop = 1;
```

```
}
```

```
remove da
```

```
{
    i < n
    i = n * 2 + 1
    dizî dışına taşıabilir!
    DİKKAT!
```

1 2 child → Olası senaryolar
 1 child → Hepsini göz önünde tut.
 3 no child

```
int remove(int n, int pQueue[])
```

```
{
    int min, i, adr;
```

```
    if (n == 0)
        return -1;
```

```
    min = pQueue[0];
    pQueue[0] ↔ pQueue[n-1];
```

```
    n--; i = 0;
    adr = findSmallChild(i, pQueue[], n);
    while ((adr > 0) && (pQueue[adr] < pQueue[i]))
```

```
{
    pQueue[i] ↔ pQueue[adr];
```

```
    i = adr;
    adr = findSmallChild(i, pQueue[], n);
```

```
}
return min;
}
```

(51)

```
int findSmallChild(int i, int pQueue[], int n)
{
    if (2*i+2 < n) → 2 child
    {
        if (pQueue[2*i+1] < pQueue[2*i+2])
            return 2*i+1;
        else
            return 2*i+2;
    }
    else if (2*i+1 < n) → 1 child
        return 2*i+1;
    else → no child
        return 0;
}
```

